Ocean University of China

Robotics Integration Group Project

Laboratory Report

Desktop Object Detection and ROS 2 Result Publishing with YOLO11n

Student: Shuyu Li

Student ID: 24020036035

School: College of Computer Science and Technology

Faculty: Faculty of Information Science and Engineering

Program: Computer Science and Technology (Sino-Foreign Cooperative Education), Class of 2024

Course: Robotics Integration Group Project

Instructor: Xiaowei Zhou

University: Ocean University of China

Project Repository:

https://github.com/BBOOOKK/summer_2026

September 2026

Abstract

This experiment develops a five-class desktop object detector and a ROS 2 result-publishing pipeline based on YOLO11n. The dataset contains 5,000 images and 8,507 bounding boxes. A 1,000-image subset derived from self-collected material originates from 10 independently captured parent photographs and comprises 10 size-normalized base images and 990 deterministic augmentations. The remaining 4,000 images were selected from Open Images V7. All derivatives of a parent photograph were assigned to the same data split to prevent source overlap between training and evaluation. After 12 epochs of fine-tuning at an input size of 640 pixels, the selected model achieved a precision of 0.8385, a recall of 0.7697, an mAP@0.5 of 0.8386, and an mAP@0.5:0.95 of 0.5735 on 650 test images. In a deterministic, class-balanced subset of 20 real Open Images test photographs, the highest-confidence predicted class was correct for all 20 images. On a Jetson Orin NX, the PyTorch FP16 model required an average of 25.951 ms per `model.predict` call, equivalent to 38.534 FPS, while the ROS 2 result topic was measured at 29.99 Hz. The implementation accepts image, video, and camera inputs, stores structured detections and low-confidence review cases, and publishes both JSON results and annotated images. The report distinguishes independent captures from derived samples and separates model-call throughput from complete node publication frequency so that every result has an explicit measurement boundary.

Keywords: object detection; YOLO11n; desktop objects; data augmentation; Jetson Orin NX; ROS 2

1 Introduction

Desktop manipulation and human-robot interaction require more than a class label. A robot also needs the location of each object so that later modules can associate detections with actions. Mice, keyboards, and laptop computers commonly occur together in a confined workspace. An open laptop introduces nested semantics because the complete device and its visible keyboard are both meaningful targets. Illumination, occlusion, viewing angle, and object pose further alter the observed appearance. A practical detector must therefore process a continuous image stream while returning class labels, bounding boxes, and confidence scores.

YOLO formulates object classification and bounding-box regression in a single forward pass and is representative of one-stage detection[1]. SSD likewise performs multi-scale prediction without an independent region-proposal stage[2]. These designs are suitable for continuous image inference when the target platform has limited computing resources. The present experiment adopts the compact YOLO11n model without altering its architecture.

The work addresses two connected tasks. First, a mixed-source dataset is constructed under a parent-image grouping rule and used to evaluate five desktop-object classes. Second, the same inference result is reused for visualization, persistent logging, and ROS 2 publication. Evaluation is divided into a 650-image detection test, a 20-image class-decision test, a Jetson camera benchmark, and a subscriber-side ROS 2 topic-frequency measurement.

The report emphasizes a reproducible chain rather than isolated successful screenshots. On the data side, it records image provenance, augmentation lineage, split assignment, and label conversion. On the model side, it fixes the weight file, thresholds, and evaluation protocol. On the deployment side, it distinguishes network inference time, complete model-call time, and the observed publication rate. These boundaries make the reported accuracy and throughput values traceable to specific files and commands.

Accordingly, the principal outcome is an end-to-end implementation covering dataset construction, model fine-tuning, offline evaluation, camera inference, error preservation, and ROS 2 integration. The purpose is not to propose a new neural-network architecture, but to demonstrate that a compact detector can be trained, measured, deployed, and documented consistently for a desktop robotics scenario.

2 Related Work

2.1 One-Stage Object Detection

The original YOLO method predicts bounding boxes and class probabilities directly from the complete image[1]. SSD places default boxes on multiple feature maps so that objects of different scales can be handled at suitable resolutions[2]. Focal Loss addresses dense-

detection imbalance through loss weighting by reducing the contribution of easy background examples[3]. These studies established complementary approaches to direct regression, multi-scale prediction, and foreground-background balancing.

Data augmentation can vary the appearance of training samples without modifying the deployed network. YOLOv4 combined color perturbation, geometric transformation, and Mosaic augmentation in its training procedure[4]. In this experiment, offline transformations use a restricted rotation range to remain close to plausible camera observations. Offline, traceable expansion is kept separate from the online augmentation applied by the training framework.

YOLO11 is distributed through the Ultralytics package, which provides consistent interfaces for training, validation, prediction, and export. The nano-scale variant is intended for lightweight inference[5]. This experiment prioritizes compatibility between the macOS training environment and the Jetson CUDA environment, together with sufficiently low model-call latency for camera use. The choice reflects these constraints and does not imply that a smaller model is universally more accurate than larger alternatives.

2.2 Data Resources, Metrics, and Robot Communication

Open Images provides natural-scene images with several forms of annotation, including a large object-detection subset[6]. This experiment uses Open Images V7 files that correspond to the five target categories[7]. Their source identifiers and original split information are preserved when boxes are converted to the local YOLO representation.

Detection quality is reported using precision, recall, mAP@0.5, and mAP@0.5:0.95. The multi-threshold metric follows the COCO convention of averaging performance over several intersection-over-union thresholds[8]; its numerical implementation is provided by the Ultralytics validation interface[9]. A stratified 20-image subset is also reported because the course acceptance criterion is stated as class-recognition accuracy rather than AP.

ROS 2 topics provide a publisher-subscriber mechanism for exchanging sensor data and robot state. Once a topic name and message type are agreed, nodes can communicate without directly invoking one another[10]. The detector can therefore remain independent while downstream software subscribes either to compact structured results or to annotated images.

3 Dataset Construction

3.1 Classes and Annotation Rules

The five classes are mouse, keyboard, laptop, cup, and headphones, with identifiers 0 through 4 in that order. Labels use the Ultralytics YOLO horizontal bounding-box format; each line stores a class identifier, normalized center coordinates, and normalized width and height[11]. An open laptop is labeled as a complete laptop, while its visible keyboard area is also labeled as keyboard. When an object is occluded, the box follows the visible portion. Open Images

annotations retain their provided box extent after class mapping and coordinate validation.

The nested laptop-keyboard rule is important for consistency. Treating a visible laptop keyboard as keyboard in one source but ignoring it in another would create contradictory supervision. The same five-class order is therefore used in the dataset configuration, model initialization, evaluation scripts, ROS 2 output, and report tables.

3.2 Source Composition and Offline Expansion

The dataset contains 1,000 self-collected-source images and 4,000 Open Images V7 images[7]. The self-collected-source portion starts from 10 independent photographs. Each parent produces one base version normalized to 1280×960 pixels and 99 deterministic augmented versions. Parameters are determined from random seed 20260831 together with the parent filename and variant number. Transformations include rotation within $[-9^\circ, 9^\circ]$, scaling within $[0.88, 1.12]$, horizontal or vertical translation of no more than 7% of the image dimension, horizontal flipping with probability 0.5, and bounded brightness, contrast, blur, and noise perturbations. Bounding boxes are transformed with the image.

The term *1,000 self-collected-source images* describes provenance, not 1,000 independent capture events. Independent visual content originates from 10 photographs; the remaining 990 files are traceable offline derivatives. Augmentation changes brightness, placement, and local imaging characteristics, but it cannot introduce a new room, camera, desk material, or capture operator. The report preserves this distinction so that derived sample count is not mistaken for independent scene count.

The public subset supplements object appearance and background variation. For every Open Images file, the manifest stores the original split, source identifier, and download URL before mapping the class name to the local identifier. If a source box is represented in pixel coordinates, conversion to YOLO format is

$$x_c = \frac{x_{\min} + x_{\max}}{2W}, \quad y_c = \frac{y_{\min} + y_{\max}}{2H}, \quad w = \frac{x_{\max} - x_{\min}}{W}, \quad h = \frac{y_{\max} - y_{\min}}{H}, \tag{1}$$

where W and H are image width and height. Converted coordinates and dimensions are then checked against the valid normalized range.

Figure 1 shows four base images and their deterministic variants.

3.3 Data Splits and Quality Control

The 10 parent photographs are assigned to training, validation, and test splits in an 8:1:1 ratio. Every derivative of a parent remains in the same split. Open Images files retain their original source split and are sampled by stable source-identifier hashes. The final training, validation, and test sets contain 4,000, 350, and 650 images, respectively. Table 1 reports source composition and box count.



Figure 1. Examples derived from self-collected material. Panels (a)–(d) are size-normalized base versions of four parent photographs; panels (e)–(h) are deterministic augmented versions with consistently transformed boxes.

Table 1. Dataset splits, image provenance, and bounding-box counts.

Split	Self-collected source	Public	Images	Boxes
Training	800	3,200	4,000	6,773
Validation	100	250	350	627
Test	100	550	650	1,107
Total	1,000	4,000	5,000	8,507

Quality control covers image readability, image-label pairing, class identifiers, normalized coordinates, duplicate source identifiers, byte-identical files, and parent-image overlap. The audit found no unreadable image, missing label, or parent-image leakage. Provenance totals are 1,000 self-collected-source images, 4,000 public images, zero generated images, and zero images of unknown origin. Perceptual-hash screening marked four cross-split pairs for visual review, but none was byte-identical and no common parent identifier crossed a split. Public annotations underwent structural checks and sampled visual inspection rather than exhaustive manual relabeling; the reported metrics therefore use the converted provider boxes as the reference annotations. File totals alone are not treated as sufficient evidence: the manifest, image tree, label tree, and source-group assignments are checked against one another.

The class distribution is shown in Table 2. Laptop has 2,549 boxes and headphones 871, producing a largest-to-smallest ratio of approximately 2.93. The offline distribution was not altered by copying a single class. Per-class precision, recall, and AP are therefore reported in addition to global means.

Table 2. Bounding-box distribution across the five classes.

Class	Boxes	Proportion (%)
mouse	1,724	20.27
keyboard	1,966	23.11
laptop	2,549	29.96
cup	1,397	16.42
headphones	871	10.24
Total	8,507	100.00

4 System Design and Implementation

4.1 Processing Pipeline

The system comprises dataset construction, model training, unified inference, and ROS 2 publication. Figure 2 summarizes the processing path. The training entry point reads the dataset configuration and produces model weights. The inference entry points accept an image, video, or USB camera stream and store annotated results and tabular records. The ROS 2 node reuses the same model call and threshold configuration before packaging detections as messages.

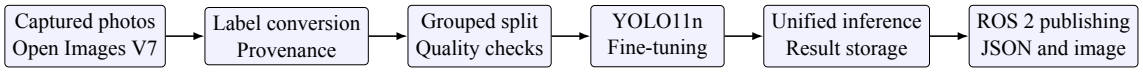


Figure 2. Dataset construction, model training, inference, and ROS 2 result-publication pipeline.

The modules share one class map and one set of inference parameters but retain separate entry points. This organization prevents camera, file, and ROS 2 modes from implementing

different post-processing logic. It also allows a failure to be located at a specific boundary: data reading, model prediction, output serialization, or message publication.

4.2 Model Fine-Tuning

The model is initialized from the previously trained five-class YOLO11n checkpoint `models/expanded_v1/best.pt`, whose class order matches the present dataset. Formal training uses 12 epochs, batch size 32, input size 640, zero data-loader workers, and early-stopping patience 4. Training runs on the Apple MPS backend with random seed 0, deterministic computation, and automatic mixed precision. The optimizer is selected automatically by Ultralytics, and the complete run configuration is preserved in `args.yaml`.

Online augmentation includes hue, saturation, and value parameters of 0.015, 0.7, and 0.4; rotation, translation, and scaling parameters are 5 degrees, 0.1, and 0.3. Horizontal flipping has probability 0.5. Mosaic starts at 0.5 and is disabled for the final 10 epochs, while MixUp is disabled. Validation metrics are computed after every epoch, and both the last checkpoint and the checkpoint with the best validation metric are saved.

Table 3 summarizes the parameters required to reproduce the run. The selected acceptance model contains 2,590,815 parameters and occupies 5,482,577 bytes. Its SHA-256 is `c53834aa239974163ef48c566dc3fea04d38de7a1d05292ab5216d5163a09a42`. Fixing the hash prevents a later checkpoint with the same `best.pt` filename from being assigned the reported results.

Table 3. Principal YOLO11n fine-tuning parameters.

Item	Setting
Initialization	`models/expanded_v1/best.pt`; unchanged five-class order
Epochs and early stopping	12 epochs; patience 4
Batch and input	batch 32; 640×640
Training device	Apple M4, MPS; workers 0
Reproducibility	seed 0; deterministic true; AMP true
Optimizer	Ultralytics auto; `lr0`=0.01; weight decay 0.0005
Geometric augmentation	degrees 5; translate 0.1; scale 0.3; horizontal flip 0.5
Composed augmentation	Mosaic 0.5, disabled for final 10 epochs; MixUp 0
Model selection	validation after each epoch; best $AP_{50:95}$ checkpoint for testing

4.3 Inference Records and ROS 2 Interface

Inference uses an input-size parameter of 640. Candidate boxes are filtered by confidence and then processed by non-maximum suppression (NMS). Each stored detection contains a class identifier, class name, confidence, and pixel-coordinate box (x_1, y_1, x_2, y_2). The same data

structure drives box drawing and CSV output for image, video, and camera input.

The confidence threshold removes low-score candidates, while NMS suppresses repeated same-class boxes that overlap a higher-confidence box beyond the configured IoU threshold. Acceptance inference uses confidence 0.50 and NMS IoU 0.45 for the 20-image test, Jetson visualization, and saved results. The offline `model.val` evaluation retains the validation interface configuration because AP is obtained from ranked detections rather than from a single presentation threshold.

The detector is also wrapped as a ROS 2 node. In camera mode, the node reads a USB device directly; in topic mode, it subscribes to `/camera/image_raw`. After each prediction, `/object_detection/result` carries JSON in `std_msgs/String`, and `/object_detection/image` carries the annotated frame in `sensor_msgs/Image`. Separating compact numerical output from the image stream lets a control process and a visualization process subscribe independently[10].

The JSON fields are listed in Table 4. Pixel-coordinate boxes can be overlaid directly on the original frame, while class identifiers support stable program logic and class names simplify terminal inspection. A frame with no accepted detection still publishes its timestamp, frame identifier, FPS value, and an empty detections array. A subscriber can therefore distinguish normal operation with no current target from a publication failure.

Table 4. Principal fields in the ROS 2 structured detection message.

Field	Form	Meaning
`timestamp`	floating point	ROS 2 node-clock timestamp in seconds
`frame_id`	integer	processed-frame counter since node startup
`fps`	floating point	exponentially smoothed model-call rate
`detections`	list	all accepted detections in the current frame
`class_id`	integer	class identifier from 0 through 4
`class_name`	string	mouse, keyboard, laptop, cup, or headphones
`confidence`	floating point	class confidence for the current box
`bbox`	object	pixel coordinates x_1, y_1, x_2, y_2

The node performs one model call per frame. JSON, the annotated image, and optional video output are derived from that result, so enabling more than one output does not duplicate neural-network inference. The result-topic queue depth is 10 and the image-topic queue depth is 2. The smaller image queue limits accumulation of large stale frames. Camera and video-writer handles are released explicitly during node shutdown.

4.4 Jetson Deployment

The acceptance checkpoint is deployed on a Jetson Orin NX. The measured run uses NVIDIA PyTorch 2.5.0, CUDA device 0, and FP16 precision. The network input is 640×640, and

confidence and NMS IoU thresholds are 0.50 and 0.45. The USB camera at `/dev/video0` delivers 640×480 frames, which Ultralytics resizes and preprocesses before inference.

TensorRT 10.7.0.23 runtime libraries are present on the device, but the Python environment used for measurement does not contain the `tensorrt` binding. The reported benchmark therefore uses the PyTorch FP16 checkpoint rather than a serialized TensorRT engine. TensorRT engines are commonly built for a target GPU and runtime environment[12, 13]. Recording installed libraries separately from the active backend prevents a software dependency from being misreported as TensorRT performance.

5 Experimental Setup and Evaluation Protocol

5.1 Training and Offline Evaluation Environment

Training and offline evaluation were conducted on macOS with an Apple M4 processor and the MPS backend. Python, PyTorch, and Ultralytics versions were 3.12.4, 2.13.0, and 8.4.133. At epoch 12, validation precision, recall, mAP@0.5, and mAP@0.5:0.95 were 0.7407, 0.7512, 0.7334, and 0.4621. The last value was the highest mAP@0.5:0.95 observed during the 12-epoch run, so this checkpoint was selected for subsequent testing.

5.2 Jetson Environment and Runtime Configuration

Edge-device tests used an NVIDIA Jetson Orin NX Engineering Reference Developer Kit Super. The device operated in `MAXN_SUPER` power mode with dynamic frequency scaling; `jetson_clocks` frequency override was not enabled. Table 5 records the measured configuration.

Table 5. Jetson hardware, software, model, and input configuration.

Field	Measured or verified value
Device and memory	NVIDIA Jetson Orin NX Engineering Reference Developer Kit Super; 16 GB; 16,031,376 kB visible to the system (approximately 15.29 GiB)
System software	JetPack 6.2.1; L4T 36.4.4; CUDA runtime 12.6.77; TensorRT libraries 10.7.0.23; ROS 2 Humble
Power configuration	`nvpmodel`: `MAXN_SUPER`, mode 0; dynamic frequency scaling, `FreqOverride=0`
Model and backend	`models/experiment1_1000self_4000public_v1/best.pt`; YOLO11n; NVIDIA-build PyTorch 2.5.0; CUDA device 0
Precision	FP16
Input	640×640 model input; HDR CAMERA-A USB camera at `/dev/video0`; 640×480 captured frames
Detection thresholds	confidence 0.50; NMS IoU 0.45

5.3 Evaluation Metrics

Let true positives, false positives, and false negatives be TP, FP, and FN. Precision and recall are

$$P = \frac{TP}{TP + FP}, \qquad R = \frac{TP}{TP + FN}. \tag{2}$$

The intersection over union between a predicted box B_p and a ground-truth box B_g is

$$\text{IoU} = \frac{|B_p \cap B_g|}{|B_p \cup B_g|}. \tag{3}$$

mAP@0.5 averages class AP at IoU 0.5. mAP@0.5:0.95 averages AP over IoU thresholds from 0.5 through 0.95 in increments of 0.05 and therefore places a stricter requirement on localization[8, 9].

5.4 Test Protocols

First, the 650-image test split was evaluated with Ultralytics 8.4.133 using `split=test`, `imgsz=640`, `batch=16`, and `device=mps`. Remaining evaluation thresholds follow that version's defaults. This run produced global and per-class metrics together with a normalized confusion matrix.

Second, a 20-image class-decision subset was formed from the public portion of the same test split. For each class, the first four Open Images files in lexicographic manifest order that contained exactly one reference box of that class were selected before inference. All 20 are real public photographs, and none belongs to the self-collected derivative groups. Predictions used confidence 0.50 and NMS IoU 0.45. A sample was counted as correct when the class of its highest-confidence box matched the manifest class; no detection or a different highest-confidence class was counted as an error. This deterministic subset is part of the 650-image test set, so its class-decision accuracy is an acceptance-oriented descriptive result rather than an independent estimate, and it is not combined with IoU-matched AP.

Third, real-time performance was measured on the Jetson Orin NX with the USB camera. Timing began after camera acquisition. The first 20 frames were used for warm-up and the next 200 for measurement. Each interval covered Ultralytics preprocessing, GPU inference, and post-processing within `model.predict`; it excluded camera reading, box drawing, video or log storage, and ROS 2 publication. The model used PyTorch FP16 at input size 640 with confidence 0.50 and NMS IoU 0.45.

Finally, ROS 2 communication performance was measured independently with `ros2 topic hz` on `/object_detection/result`. The sliding window contained 200 messages and the observation interval was 15 s. This frequency represents messages received by the subscriber and differs from the exponentially smoothed model-call FPS embedded in the JSON. Per-frame benchmark measurements are stored on the Jetson at `outputs/`

`benchmark/jetson_orin_nx_camera.csv`.

5.5 Reproduction Materials and Result Attribution

Key artifacts are stored by experiment stage, as summarized in Table 6. The dataset configuration defines paths and class order; the manifest records per-image provenance; `args.yaml` fixes training parameters; evaluation JSON and CSV files store measurements; and the review directory preserves low-confidence or failed examples produced by the test routine. The reported 650-image metrics and 20-image results are taken from the directory named for the current experiment rather than from earlier generic outputs retained elsewhere in the repository.

This structure addresses a common ambiguity during iterative experiments: identical filenames associated with different checkpoints. A reported value is attributed only after identifying model hash, data version, thresholds, and execution device. The Jetson timing boundary is also fixed by code. Camera acquisition precedes the timer, while drawing and ROS 2 publication follow it. Thus 25.951 ms is correctly described as a complete `model.predict` call, not as latency from camera exposure to subscriber display.

Table 6. Principal reproduction materials and their roles.

Material	Repository location or record
Project repository	https://github.com/BBOOOKK/summer_2026
Dataset configuration	`datasets/experiment1_1000self_4000public_v1/dataset.yaml`
Provenance manifest	`datasets/experiment1_1000self_4000public_v1/manifest.csv`
Provenance audit	`datasets/experiment1_1000self_4000public_v1/qa/provenance_audit.json`
Training configuration	`outputs/training/experiment1_1000self_4000public_v1/args.yaml` and `results.csv`
Test-set metrics	`outputs/evaluation/experiment1_1000self_4000public_v1_test/evaluation_results.json`
20-image selection and results	`configs/test_samples.csv` and `20_object_test_results.csv` in the experiment evaluation directory
Low-confidence review cases	`outputs/errors/experiment1_1000self_4000public_v1_real20/`
Jetson measurement record	`docs/JETSON_MEASUREMENT.md`; per-frame CSV retained on the Jetson
Acceptance checkpoint	`models/experiment1_1000self_4000public_v1/best.pt` and its SHA-256

6 Results and Analysis

6.1 Training Behavior and Overall Detection Results

Figure 3 shows the 12-epoch run. All three training losses decrease overall, whereas validation losses fluctuate. Validation mAP@0.5 and mAP@0.5:0.95 reach 0.7334 and 0.4621 at epoch 12, and no clear performance plateau is observed within the scheduled training interval.

From epoch 1 to epoch 12, training box loss decreases from 0.9981 to 0.7168, classification loss from 0.9210 to 0.5428, and distribution focal loss from 1.3143 to 1.0812. Over the same interval, validation box loss and classification loss decrease from 1.4159 and 1.2833 to 1.1461 and 1.0796, whereas validation distribution focal loss changes from 1.7542 to 1.7939 with repeated fluctuations. Training-set convergence is therefore more regular than validation localization behavior.

Validation mAP@0.5 increases from 0.6760 at epoch 1 to 0.7334 at epoch 12, while mAP@0.5:0.95 increases from 0.3804 to 0.4621. The final three mAP@0.5:0.95 values are 0.4476, 0.4450, and 0.4621; the selected checkpoint is therefore based on the highest observed validation AP rather than on the training-loss minimum alone. Total training time was 3,467.77 s, approximately 57.8 min. This value describes the present run and is not used as a cross-hardware training benchmark.

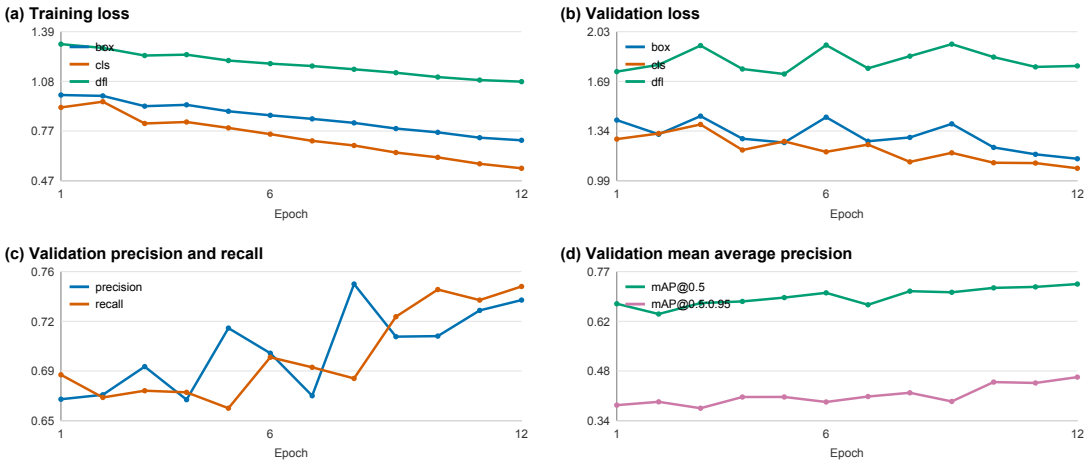


Figure 3. YOLO11n fine-tuning history: (a) training losses; (b) validation losses; (c) validation precision and recall; and (d) validation mean average precision.

Detection results on the 650-image test set are reported in Table 7. Global precision is 0.8385 and recall is 0.7697, indicating that retained predictions are more reliable than the detector's coverage of all ground-truth objects. mAP@0.5 is 0.8386 and mAP@0.5:0.95 is 0.5735. Their difference of 0.2651 shows a substantial reduction when stricter IoU conditions are introduced, so boundary localization remains more sensitive than class recognition at IoU 0.5.

Table 7. Global and per-class detection metrics on the test set.

Class	P	R	AP_{50}	$AP_{50:95}$
mouse	0.9325	0.9101	0.9501	0.7546
keyboard	0.8346	0.8469	0.9012	0.6361
laptop	0.9121	0.7529	0.8640	0.4599
cup	0.9091	0.7409	0.8571	0.6751
headphones	0.6041	0.5978	0.6203	0.3419
Overall	0.8385	0.7697	0.8386	0.5735

6.2 Class Differences and Error Patterns

Mouse has the strongest result, with $AP_{50} = 0.9501$ and recall 0.9101. Keyboard includes both standalone keyboards and visible laptop-keyboard regions, yet reaches $AP_{50} = 0.9012$. Laptop precision is 0.9121, but recall is 0.7529 and $AP_{50:95}$ decreases to 0.4599, indicating weaker target coverage and stricter localization.

Headphones has the lowest precision, recall, and AP_{50}, at 0.6041, 0.5978, and 0.6203. It also has the fewest boxes and appears in open, folded, and partially occluded configurations. These observations show that lower sample count and weaker performance co-occur; they do not establish a direct causal relationship.

Cup precision is 0.9091 while recall is 0.7409. Accepted cup predictions are therefore usually correct, but a portion of ground-truth objects remain unmatched. Keyboard precision and recall are closer, at 0.8346 and 0.8469. These metric combinations describe different failure tendencies: lower precision is associated with more false positives, lower recall with more missed targets, and a wide AP_{50} to $AP_{50:95}$ gap with reduced localization stability under strict overlap criteria.

For laptop, AP_{50} is 0.8640 but $AP_{50:95}$ is 0.4599, a difference of 0.4041. An open laptop contains nested complete-device and visible-keyboard boxes, whereas a closed laptop presents a wide, low silhouette. These annotation semantics and appearance changes may contribute to localization complexity. The interpretation is consistent with inspected samples, but pose-specific AP was not computed and no direct causal claim is made.

The normalized confusion matrix is shown in Figure 4. With columns normalized by true class, diagonal values for mouse, keyboard, laptop, cup, and headphones are 0.93, 0.89, 0.82, 0.78, and 0.67. In the background row, cup and headphones reach 0.22 and 0.32, indicating more unmatched ground-truth targets for these classes. Explicit class-to-class off-diagonal confusion is comparatively limited, consistent with the lower recall of cup and headphones.

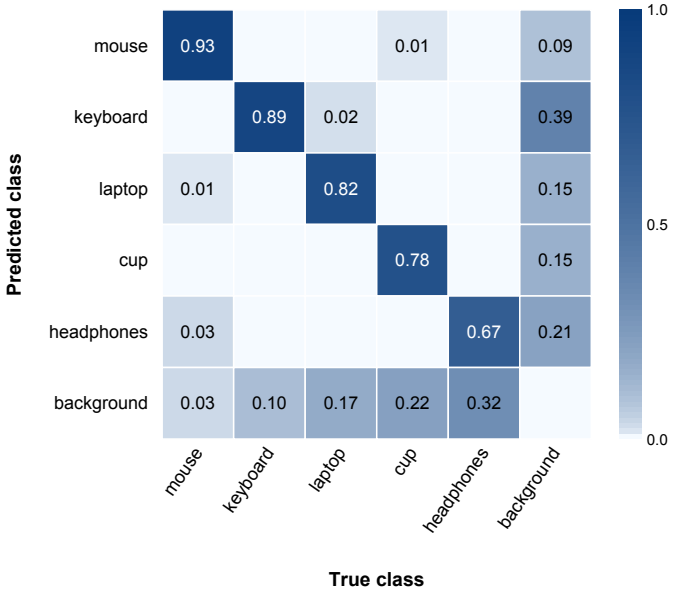


Figure 4. Normalized confusion matrix on the 650-image test split. Columns are true classes and rows are predicted classes.

The 20-image script also preserves any prediction below a review threshold of 0.60 even when its class decision is correct. Three such cases occur in the deterministic subset: two keyboards and one laptop. They are retained as inspectable low-confidence examples, not as estimates of the frequency of a particular failure mechanism. Missed detections, misclassifications, and false positives would be stored by the same routine when present.

6.3 Twenty-Image Class Decisions and Multi-Class Output

For the deterministic 20-image subset, the highest-confidence predicted class matches the reference class for all 20 photographs, giving 100% class-decision accuracy and exceeding the required 80%. Because these images are drawn from the evaluated 650-image test split, this subset result describes a fixed acceptance sample rather than an additional independent test.

Per-class results are shown in Table 8. Every class achieves 4/4. Mean confidence is calculated over the four accepted class decisions for each category and is included to describe the score range, not as a calibrated probability or global reliability estimate. Keyboard has the lowest mean confidence, 0.6543, and includes the two lowest scores, 0.5083 and 0.5654; both remain above the configured confidence threshold.

To examine simultaneous multi-class output, the current acceptance checkpoint was applied to `self_IMG_8306_v000.jpg`, a base image in the self-collected-source portion of the test split. Inference on the training machine used MPS with confidence 0.50 and NMS IoU 0.45. The image contains laptop, keyboard, and mouse ground-truth objects; all three are detected in the same frame with confidence 0.9233, 0.9184, and 0.8631. Figure 5 shows the complete output.

Table 8. Per-class decisions on the 20 single-object samples.

Class	Correct/total	Accuracy (%)	Mean confidence[a]	Range[a]
mouse	4/4	100	0.9214	0.8609–0.9642
keyboard	4/4	100	0.6543	0.5083–0.8501
laptop	4/4	100	0.7774	0.5904–0.9089
cup	4/4	100	0.8881	0.7531–0.9595
headphones	4/4	100	0.8038	0.7140–0.8703
Overall	20/20	100	–	–

[a]Calculated over the four accepted class decisions in each row.

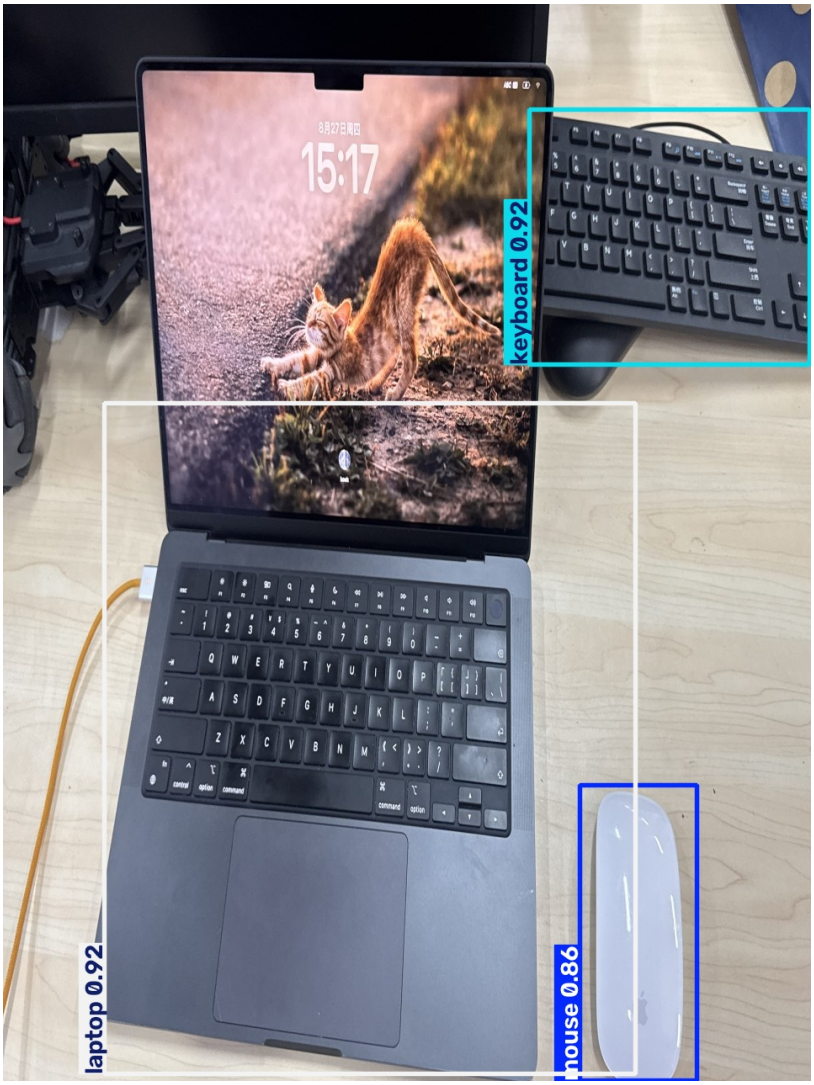


Figure 5. Multi-class output from the current acceptance checkpoint on a base test image derived from self-collected material. Laptop, keyboard, and mouse are detected simultaneously with bounding boxes and confidence values.

6.4 Jetson Real-Time Detection and ROS 2 Publication

After 20 warm-up frames, the Jetson benchmark measured 200 camera frames. Mean `model.predict` call time was 25.951 ms, corresponding to 38.534 FPS, and the minimum per-frame call rate was 37.952 FPS. Ultralytics reported mean network inference time of 20.575 ms and maximum network inference time of 20.896 ms. The 5.376 ms difference between mean model-call time and mean network time contains preprocessing, post-processing, and prediction-interface overhead. Mean model-call throughput is 7.71 times the 5 FPS acceptance threshold, and the minimum observed call rate also exceeds the threshold.

Figure 6 contains 32 chronologically selected frames from a recording of the browser-displayed Jetson detection stream. The sequence covers 0.25–15.60 s and is arranged from left to right and top to bottom. As the view moves across the desktop, laptop, keyboard, and mouse labels, confidence values, and boxes appear continuously. FPS displayed in the upper-left corner is the program's exponentially smoothed model-call rate for runtime observation; the formal performance value remains the fixed warm-up and measurement benchmark.

The observed publication frequency of `/object_detection/result` is 29.99 Hz over a 15 s window, close to the approximately 30 FPS camera input. It is lower than the independent model-call benchmark because continuous node execution also includes frame acquisition, JSON serialization, box drawing, and image-message conversion. Both measurements exceed 5 FPS, showing that model inference and ROS 2 result publication satisfy the real-time requirement in the measured configuration. Table 9 consolidates the principal results.

Table 9. Summary of detection results and Jetson performance.

Test item	Result
Test-set size	650 images; 1,107 boxes
Overall mAP@0.5 / mAP@0.5:0.95	0.8386 / 0.5735
20-image class-decision accuracy	20/20; 100%
Jetson backend	PyTorch FP16; CUDA device 0
Jetson warm-up/measured frames	20/200
Mean network inference time	20.575 ms
Mean `model.predict` call time	25.951 ms
Mean/minimum model-call rate	38.534/37.952 FPS
Maximum network inference time	20.896 ms
ROS 2 result-topic frequency	29.99 Hz (15 s; window 200)
Real-time requirement and decision	≥ 5 FPS; PASS

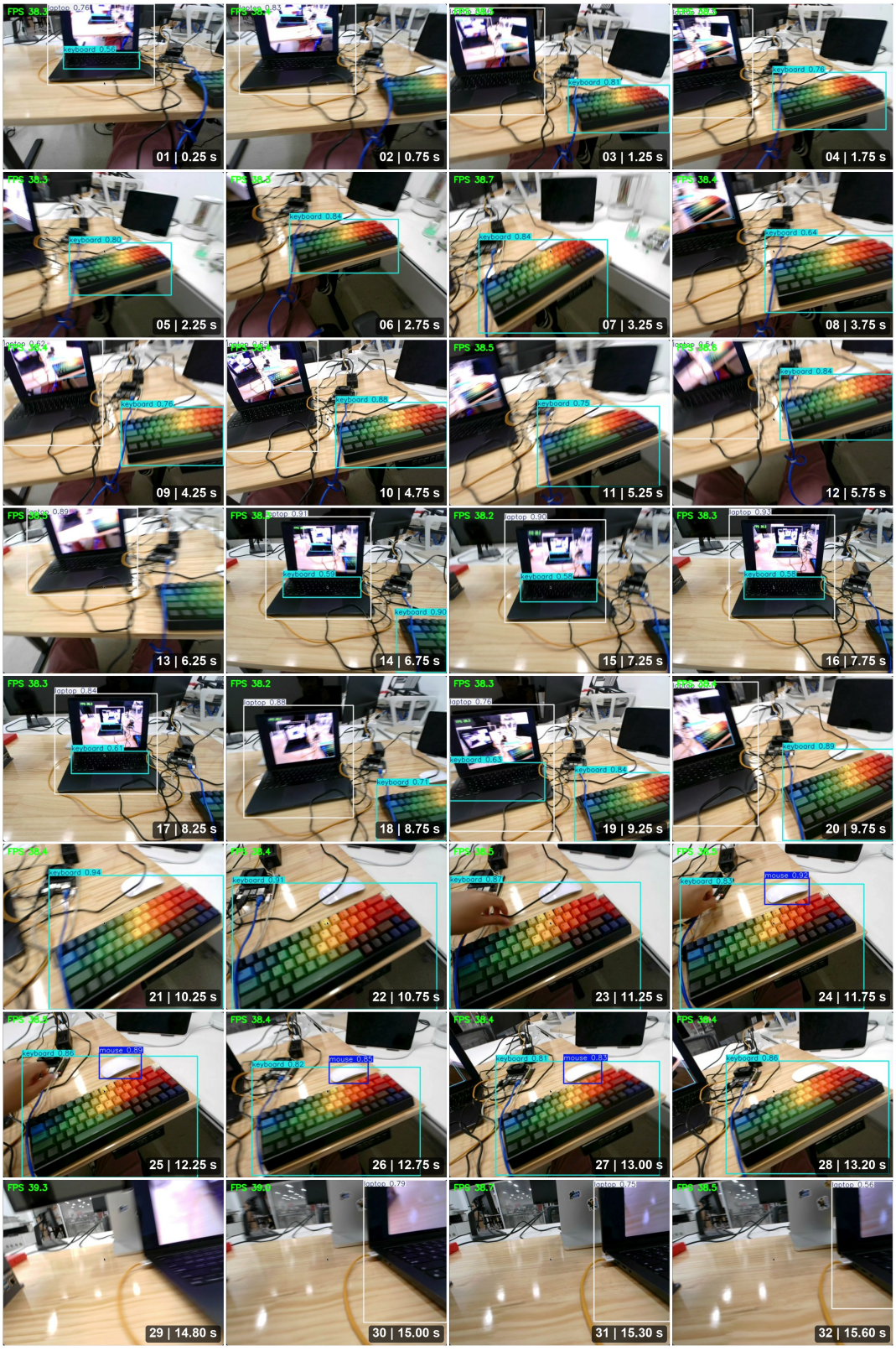


Figure 6. Thirty-two frames from the Jetson real-time detection recording, arranged chronologically from 0.25 to 15.60 s. Each panel retains class labels, confidence values, bounding boxes, and the program-displayed exponentially smoothed model-call FPS.

6.5 Integrated Interpretation of the Evaluation Results

The reported measurements answer different questions and must not be substituted for one another. Test-set AP depends on IoU matching between predictions and reference boxes and evaluates ranking and localization over multiple images and objects. The 20-image value compares only each image's highest-confidence class with its manifest class and corresponds to the course's class-recognition criterion. Its 100% value does not imply a test-set recall of 1.00 because the subset is small, its class composition is fixed, its images are already part of the test split, and its matching rule differs from detection AP.

Jetson throughput of 38.534 FPS is obtained from a fixed 20-frame warm-up and 200 timed `model.predict` calls. The 29.99 Hz ROS 2 result is observed at the subscriber over 15 s. The difference of approximately 8.54 is consistent with the broader work performed by the continuous node. Relative to the 5 FPS requirement, mean model-call throughput is 7.71 times the threshold and topic frequency is approximately 6.00 times the threshold. The report retains both values instead of replacing either with the screen recording's container frame rate.

The multi-class base image and 32-frame Jetson sequence provide qualitative evidence. The former verifies that laptop, keyboard, and mouse can be produced in one frame, while the sampled sequence is consistent with sustained annotated output as the camera view changes. The original continuous recording supports temporal inspection; neither the still image nor the montage independently establishes aggregate accuracy or minimum throughput. Numerical metrics, stored examples, and runtime video therefore serve distinct evidential roles.

References

[1] J. Redmon, S. Divvala, R. Girshick, and A. Farhadi, "You only look once: Unified, real-time object detection," in *Proceedings of the IEEE Conference on Computer Vision and Pattern Recognition*, 2016, pp. 779–788.

[2] W. Liu, D. Anguelov, D. Erhan, C. Szegedy, S. Reed, C.-Y. Fu, and A. C. Berg, "SSD: Single shot MultiBox detector," in *Computer Vision – ECCV 2016*, ser. Lecture Notes in Computer Science, vol. 9905, 2016, pp. 21–37.

[3] T.-Y. Lin, P. Goyal, R. Girshick, K. He, and P. Dollár, "Focal loss for dense object detection," in *Proceedings of the IEEE International Conference on Computer Vision*, 2017, pp. 2980–2988.

[4] A. Bochkovskiy, C.-Y. Wang, and H.-Y. M. Liao, "YOLOv4: Optimal speed and accuracy of object detection," *arXiv preprint arXiv:2004.10934*, 2020.

[5] G. Jocher and J. Qiu, "Ultralytics YOLO11," 2024, accessed: Sep. 5, 2026. [Online]. Available: https://github.com/ultralytics/ultralytics

[6] A. Kuznetsova, H. Rom, N. Alldrin, J. Uijlings, I. Krasin, J. Pont-Tuset, S. Kamali, S. Popov, M. Malloci, A. Kolesnikov, T. Duerig, and V. Ferrari, "The Open Images Dataset V4: Unified image classification, object detection, and visual relationship detection at scale," *International Journal of Computer Vision*, vol. 128, no. 7, pp. 1956–1981, 2020.

[7] Google, "Open Images Dataset V7 and extensions," 2022, accessed: Sep. 5, 2026. [Online]. Available: https://storage.googleapis.com/openimages/web/index.html

[8] T.-Y. Lin, M. Maire, S. Belongie, L. Bourdev, R. Girshick, J. Hays, P. Perona, D. Ramanan, C. L. Zitnick, and P. Dollár, "Microsoft COCO: Common objects in context," in *Computer Vision – ECCV 2014*, 2014, pp. 740–755.

[9] Ultralytics, "Model validation with Ultralytics YOLO," 2026, accessed: Sep. 5, 2026. [Online]. Available: https://docs.ultralytics.com/modes/val/

[10] Open Robotics, "ROS 2 Humble documentation: Topics," 2026, accessed: Sep. 5, 2026. [Online]. Available: https://docs.ros.org/en/humble/Concepts/Basic/About-Topics.html

[11] Ultralytics, "Object detection datasets overview," 2026, accessed: Sep. 5, 2026. [Online]. Available: https://docs.ultralytics.com/datasets/detect/

[12] NVIDIA Corporation, "How TensorRT works," 2026, TensorRT 11.2.1 documentation; Accessed: Sep. 5, 2026. [Online]. Available: https://docs.nvidia.com/deeplearning/tensorrt/latest/architecture/how-trt-works.html

[13] ——, "TensorRT engine compatibility," 2026, TensorRT 11.2.1 documentation; Accessed: Sep. 5, 2026. [Online]. Available: https://docs.nvidia.com/deeplearning/tensorrt/latest/inference-library/engine-compatibility.html